

Backend · Portfolio & Git / GitHub

[Topic 1](#) · [Topic 2](#) · [Topic 3](#) | [Complete Study Guide](#)

[Backend Dependencies](#) · [Building a Portfolio](#) · [Git & GitHub Mastery](#)

TOPIC 1

Backend Dependencies for Applications

Backend dependencies are external libraries, frameworks, and tools that a server-side application relies on to function. They are declared in a manifest file (package.json, requirements.txt, pom.xml, etc.) and installed via a package manager. Understanding the categories of dependencies and choosing the right ones is a core backend engineering skill.

1. What Are Dependencies?

Term	Definition
Dependency	External code your project imports and uses at runtime or build time.
Package Manager	Tool that downloads, installs, and manages dependencies automatically.
Manifest File	Declares which packages and versions your project needs.
Lock File	Records exact resolved versions to ensure reproducible installs.
Transitive Dep	A dependency of your dependency — pulled in automatically.

2. Package Managers by Language

Language	Package Manager	Config File
Node.js	npm / yarn / pnpm	package.json + package-lock.json
Python	pip / poetry / conda	requirements.txt / pyproject.toml
Java	Maven / Gradle	pom.xml / build.gradle
PHP	Composer	composer.json
Ruby	Bundler (gem)	Gemfile + Gemfile.lock
Go	Go Modules	go.mod + go.sum
Rust	Cargo	Cargo.toml + Cargo.lock
.NET / C#	NuGet	.csproj / packages.config

3. Categories of Backend Dependencies

A. Web Frameworks — Handling HTTP Requests & Routing		
Library/Framework	Language	Purpose
Express.js	Node.js	Minimalist, highly flexible, most popular Node framework.

FastAPI	Python	Modern async framework; auto-generates OpenAPI docs.
Django	Python	Batteries-included: ORM, admin, auth out of the box.
Flask	Python	Lightweight microframework; minimal boilerplate.
Spring Boot	Java	Enterprise-grade; convention-over-configuration.
NestJS	Node.js	Angular-inspired structure with TypeScript; great for large APIs.
Laravel	PHP	Full-stack MVC framework with elegant syntax.
Rails	Ruby	Opinionated MVC; rapid development with scaffolding.
Gin	Go	High-performance HTTP router; minimal overhead.
ASP.NET Core	.NET	Cross-platform, high-performance Microsoft framework.

B. Database & ORM Dependencies

Library	Ecosystem	Purpose
Mongoose	Node/MongoDB	ODM for MongoDB — schema validation and querying.
Sequelize	Node/SQL	Promise-based ORM for PostgreSQL, MySQL, SQLite.
Prisma	Node/TypeScript	Type-safe ORM with auto-generated client from schema.
SQLAlchemy	Python	Comprehensive ORM + SQL expression language.
Tortoise ORM	Python async	Async ORM inspired by Django for Python 3.
Hibernate	Java	Gold-standard JPA ORM; powerful but verbose.
TypeORM	TypeScript	Decorator-based ORM; works with many SQL databases.
pg / mysql2	Node	Raw database driver — used when you want full SQL control.
Redis (ioredis)	Node/Python	In-memory key-value store for caching & pub/sub.
Elasticsearch	Multi	Full-text search and analytics engine client library.

C. Authentication & Security Dependencies

Library	Ecosystem	Purpose
jsonwebtoken	Node.js	Sign and verify JWT tokens for stateless auth.

bcrypt / bcryptjs	Node/Python	Hash and compare passwords securely.
passport.js	Node.js	Auth middleware with 500+ strategies (OAuth, local, JWT).
PyJWT	Python	JWT encode/decode library for Python.
python-jose	Python	JOSE / JWT library supporting RS256 and more.
Spring Security	Java	Comprehensive auth, CORS, CSRF, and access control.
helmet	Node.js	Sets secure HTTP headers (XSS, clickjacking protection).
cors	Node.js	Configure Cross-Origin Resource Sharing policies.
express-rate-limit	Node	Rate limiting middleware to prevent brute-force attacks.
cryptography	Python	Cryptographic primitives — AES, RSA, Fernet encryption.

D. API & Communication Dependencies

Library	Ecosystem	Purpose
axios	Node.js	Promise-based HTTP client for calling external APIs.
httpx	Python	Async-first HTTP client with sync support.
requests	Python	Simple, elegant synchronous HTTP client.
socket.io	Node.js	Real-time bidirectional communication via WebSockets.
graphql-js	Node.js	Reference GraphQL implementation for building APIs.
Apollo Server	Node.js	Full-featured GraphQL server with subscriptions.
Strawberry	Python	Modern GraphQL library using Python type hints.
grpc	Multi	High-performance RPC framework client/server library.
celery	Python	Distributed task queue for async background jobs.
Bull / BullMQ	Node.js	Redis-based queue for background job processing.

E. Testing Dependencies

Library	Ecosystem	Purpose
Jest	Node.js	All-in-one test runner, assertion library, and mocking.
Mocha + Chai	Node.js	Flexible test runner with expressive assertions.
Supertest	Node.js	HTTP integration testing for Express APIs.

pytest	Python	Powerful, simple test framework with fixture support.
unittest	Python	Built-in standard library testing module.
JUnit 5	Java	Standard unit testing framework for JVM.
Mockito	Java	Mocking framework for isolating units under test.
Vitest	Node.js	Vite-native test framework; blazing-fast alternative to Jest.

F. Utility & Validation Dependencies

Library	Ecosystem	Purpose
lodash	Node.js	Utility belt for arrays, objects, strings, functions.
moment / dayjs	Node.js	Date parsing, formatting, and manipulation.
zod	TypeScript	Schema validation with TypeScript inference.
joi	Node.js	Object schema description and validation.
pydantic	Python	Data validation using Python type annotations.
dotenv	Node.js	Load environment variables from .env files.
winston / pino	Node.js	Structured logging with transports and log levels.
loguru	Python	User-friendly structured logging for Python.
multer	Node.js	Handle multipart/form-data (file uploads).
uuid	Multi	Generate unique identifiers (v1, v4, v5).

4. Dependency Files — What They Look Like

Node.js — package.json

```
{
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^8.1.0",
    "jsonwebtoken": "^9.0.0",
    "bcryptjs": "^2.4.3"
  },
  "devDependencies": {
    "jest": "^29.0.0",
    "nodemon": "^3.0.0"
  }
}
```

Python — requirements.txt

```
fastapi==0.110.0
uvicorn==0.27.0
sqlalchemy==2.0.27
pydantic==2.6.1
python-jose==3.3.0
bcrypt==4.1.2
pytest==8.0.0
```

5. Dependency Best Practices

- Pin versions in production (==) but allow minor updates in dev (^/~).
- Regularly audit dependencies: npm audit / pip-audit / snyk.
- Separate dev dependencies from production dependencies.
- Use lock files (package-lock.json, poetry.lock) in version control.
- Avoid unused dependencies — they increase attack surface and bundle size.
- Prefer well-maintained packages with active contributors and recent releases.
- Use environment variables (.env) to inject secrets — never hard-code API keys.

TOPIC 2

Build a Portfolio on Your Own

A developer portfolio is your professional showcase — it demonstrates your skills, projects, and personality to recruiters, clients, and collaborators. A well-built portfolio often matters more than a resume alone.

1. Why a Portfolio Matters

- Proves you can build real things, not just answer interview questions.
- Shows code quality, design sense, and ability to ship end-to-end.
- Differentiates you from candidates with identical resumes.
- Acts as a living document — grows with your career.
- GitHub + Portfolio = your developer identity online.

2. What to Include

Section	What to Put Here
Hero / Introduction	Your name, role (e.g. Full Stack Developer), 1-line tagline, and CTA buttons (GitHub, Resume, Contact).
About Section	Short bio, what you build, your tech stack, and what drives you.
Skills / Tech Stack	Grouped icons or tags: Languages, Frameworks, Tools, Databases.
Projects Section	3–6 featured projects with title, description, tech used, live link, and GitHub link.
Experience / Timeline	Internships, jobs, freelance work — role, company, and key achievements.
Education	Degree, institution, graduation year, relevant coursework.
Certifications	AWS, Google Cloud, Meta, Coursera — adds credibility.
Blog / Articles	Optional — shows depth of knowledge and communication skills.
Contact Section	Email form or direct email, LinkedIn, GitHub, Twitter/X links.

3. Portfolio Project Ideas (For Students)

Project	Tech Stack	Skills Demonstrated
Todo / Task Manager	React + Node + MongoDB	CRUD, auth, RESTful API basics.

Weather App	React + OpenWeather API	API integration, async fetch, UI.
Blog Platform	Next.js + PostgreSQL	SSR, CMS, markdown, SEO.
E-Commerce Store	React + Stripe + Node	Payments, cart, auth, full-stack.
Chat Application	Node + Socket.io	Real-time WebSockets, rooms.
URL Shortener	FastAPI + Redis	Backend logic, caching, databases.
Portfolio Site Itself	HTML/CSS or React	Shows design + frontend skills.
ML Model API	Python + FastAPI	AI integration, model serving.

4. Step-by-Step: Building Your Portfolio

Step 1 — Plan & Design

- Choose a style: minimal, creative, dark-themed, or card-based.
- Sketch a wireframe on paper or Figma before coding.
- Pick your color palette (2–3 colors) and font pairing.
- Decide on sections order: Hero → About → Skills → Projects → Contact.

Step 2 — Set Up Your Project

```
# Option A: Pure HTML/CSS/JS (beginner-friendly)
mkdir my-portfolio && cd my-portfolio
touch index.html style.css script.js

# Option B: React (recommended for interactivity)
npx create-react-app my-portfolio

# OR with Vite (faster):
npm create vite@latest my-portfolio -- --template react
cd my-portfolio && npm install && npm run dev
```

Step 3 — Build Section by Section

- **Navbar** – Logo/name + smooth-scroll links to each section.
- **Hero** – Full-screen with your name, title, animated text (Typed.js), and buttons.
- **About** – Photo + short bio paragraph + downloadable resume link.
- **Skills** – Grid of icons (use devicons.dev for tech logos).
- **Projects** – Cards with image/screenshot, description, tag badges, GitHub + Live links.
- **Contact** – EmailJS or Formspree for a working contact form without a backend.

Step 4 — Make It Responsive

```
/* CSS approach */
@media (max-width: 768px) {
  .projects-grid { grid-template-columns: 1fr; }
  .hero-text { font-size: 2rem; }
}
```


Use CSS Grid and Flexbox for layout. Test on mobile, tablet, and desktop viewports using Chrome DevTools.

Step 5 — Add Animations

- AOS (Animate On Scroll) — fade-in sections as user scrolls.
- Framer Motion — React animation library for smooth transitions.
- CSS transitions on hover for buttons and project cards.
- Particles.js or Vanta.js for animated hero backgrounds.

Step 6 — Deploy for Free

Platform	Best For
GitHub Pages	Free static hosting. Push code, enable Pages in repo settings.
Vercel	Best for React/Next.js. Connect GitHub repo — auto-deploys on push.
Netlify	Drag & drop deploy or GitHub integration. Free SSL certificate.
Render	Good for full-stack apps with backend (Node, Python).
Cloudflare Pages	Fast CDN, free tier with unlimited requests.

```
# Deploy to GitHub Pages (simplest for HTML sites)
git init && git add . && git commit -m 'Initial portfolio'
git remote add origin https://github.com/username/portfolio.git
git push -u origin main
# Then: Settings -> Pages -> Source: main branch
```

5. Portfolio Checklist

- Custom domain (yourname.dev / .com) — ~\$10/year from Namecheap or Google Domains.
- SEO meta tags: title, description, og:image for social sharing previews.
- Favicon — small icon shown in browser tab.
- Google Analytics or Plausible to track visitors.
- All project links are live and working.
- Resume PDF link opens or downloads correctly.
- Form submissions deliver to your email.
- Lighthouse score above 90 for Performance, Accessibility, SEO.
- No spelling errors — proofread everything.

- GitHub profile README links back to your portfolio.

TOPIC 3

Git and GitHub

Git is a **distributed version control system** that tracks changes in source code. GitHub is a cloud-based platform built around Git that adds collaboration, code review, CI/CD, and project management features. Together they are the backbone of modern software development.

1. Git vs GitHub

Tool	What It Is
Git	A local version control tool. Runs on your machine. Tracks history of files.
GitHub	A cloud platform hosting Git repositories. Adds collaboration, PRs, Issues, Actions.

2. Core Git Concepts

Concept	Definition
Repository (Repo)	A project folder tracked by Git. Contains all files + history (.git folder).
Commit	A snapshot of your changes at a point in time. Has a unique SHA hash.
Branch	An independent line of development. main is the default branch.
Merge	Combining changes from one branch into another.
Clone	Copy a remote repository to your local machine.
Pull	Fetch + merge latest changes from remote to local.
Push	Upload your local commits to the remote repository.
Fork	Personal copy of someone else's repository on GitHub.
Pull Request (PR)	A request to merge your branch/fork changes into the main codebase.
Staging Area	Intermediate area where you add files before committing.
HEAD	Pointer to the current commit/branch you are working on.
.gitignore	File listing patterns of files Git should NOT track (e.g. node_modules).

3. Essential Git Commands

Setup — First Time Configuration

```
git config --global user.name 'Your Name'
```

```
git config --global user.email 'you@email.com'  
git config --global core.editor 'code --wait' # set VS Code as default editor
```

Starting a Repository

```
git init # initialise new repo in current folder  
git clone # clone remote repo to local machine
```

Tracking Changes

```
git status # show working tree status  
git add filename.js # stage a specific file  
git add . # stage all changes  
git add -p # interactively stage chunks of changes  
git diff # show unstaged changes  
git diff --staged # show staged changes
```

Committing

```
git commit -m 'feat: add login endpoint' # commit with message  
git commit --amend # edit last commit message  
git commit -am 'fix: correct typo' # stage tracked files + commit
```

Branching

```
git branch # list all local branches  
git branch feature/login # create a new branch  
git checkout feature/login # switch to branch  
git checkout -b feature/login # create AND switch  
git switch -c feature/login # modern alternative to checkout -b  
git branch -d feature/login # delete branch (after merge)
```

Merging & Rebasing

```
git merge feature/login # merge branch into current branch  
git rebase main # replay commits on top of main (cleaner history)  
git merge --no-ff feature # merge with a merge commit (preserves branch history)
```

Remote Operations

```
git remote -v # show remote connections  
git remote add origin # add remote named 'origin'  
git push origin main # push local main to remote  
git push -u origin feature/login # push branch and set upstream  
git pull origin main # fetch + merge from remote  
git fetch origin # download changes (don't merge yet)
```

Undoing Changes

```
git restore filename.js # discard unstaged changes to a file  
git restore --staged file # unstage a file  
git reset HEAD~1 # undo last commit (keep changes staged)  
git reset --hard HEAD~1 # undo last commit (DISCARD changes)  
git revert # create new commit that undoes a previous one
```

```
git stash # temporarily save uncommitted changes
git stash pop # restore stashed changes
```

Viewing History

```
git log # full commit history
git log --oneline --graph # compact visual branch graph
git log -p # show diff for each commit
git show # details of a specific commit
git blame filename.js # who last changed each line
```

4. Git Workflow — Feature Branch Workflow

- 1. Start from updated main: git checkout main && git pull
- 2. Create feature branch: git checkout -b feature/your-feature
- 3. Make changes and commit: git add . && git commit -m 'feat: ...'
- 4. Push branch to GitHub: git push -u origin feature/your-feature
- 5. Open Pull Request on GitHub for code review.
- 6. Address review feedback, push more commits to same branch.
- 7. Merge PR after approval. Delete feature branch.
- 8. Pull latest main locally: git checkout main && git pull

5. Commit Message Best Practices — Conventional Commits

A widely adopted standard for writing clear, machine-readable commit messages:

Prefix	Usage
feat:	A new feature. feat: add user registration endpoint
fix:	A bug fix. fix: resolve JWT expiry not being checked
docs:	Documentation changes. docs: update README setup steps
style:	Formatting, missing semicolons (no logic change).
refactor:	Code change that neither fixes a bug nor adds a feature.
test:	Adding or updating tests. test: add auth middleware tests
chore:	Build process, tooling, dependency updates.
perf:	Performance improvement. perf: cache DB query results
ci:	CI/CD config changes. ci: add GitHub Actions deploy workflow

6. GitHub Features

Repositories

- **Public** – Visible to everyone. Great for open source and portfolio projects.
- **Private** – Only you and collaborators can see it.
- **README.md** – Markdown file displayed on repo homepage. Explain what the project does.
- **Topics & Description** – Help others discover your project.

Pull Requests (PRs)

- Core collaboration unit — propose changes, discuss, request reviews.
- Linked to Issues: 'Closes #42' in PR description auto-closes the issue on merge.
- Review types: Comment, Approve, Request Changes.
- Required reviews can be enforced via branch protection rules.

Issues

- Track bugs, feature requests, and tasks.
- Label system: bug, enhancement, good first issue, help wanted, etc.
- Milestones group issues into releases or sprints.
- Assign issues to team members for accountability.

GitHub Actions — CI/CD

GitHub Actions automates workflows triggered by events (push, PR, schedule).

```
# .github/workflows/test.yml
name: Run Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm install
      - run: npm test
```

Common uses: Run tests on every PR, deploy to Vercel/AWS on merge to main, lint and format checks, publish npm packages, send Slack notifications.

GitHub Pages

- Free static site hosting directly from a repository.
- Enable in Settings → Pages → Source branch.
- Perfect for portfolio sites, documentation, and project demos.
- Custom domain support with automatic HTTPS.

7. .gitignore — What to Exclude

```
# Node.js
node_modules/
.env
dist/

# Python
__pycache__/
*.pyc
.venv/
.env

# General
.DS_Store # macOS
Thumbs.db # Windows
*.log
*.tmp
```

Visit gitignore.io to generate .gitignore files for any language or framework automatically.

8. GitHub Profile — Make It Count

- Profile README: Create a repo named exactly your username — its README.md shows on your profile.
- Pin 6 best repositories on your profile page.
- Green contribution graph — commit daily, even small improvements count.
- Star repos you use — shows your interests to visitors.
- Follow developers in your field to build your network.
- Add bio, location, website (your portfolio), and Twitter/X handle.
- Write good READMEs: project description, screenshots, setup instructions, tech stack.

9. Quick Reference Cheat Sheet

Action	Command	Purpose
Init repo	git init	Create new local repo
Clone	git clone	Copy remote repo
Stage all	git add .	Stage all changes
Commit	git commit -m 'message'	Save snapshot
Push	git push origin branch	Upload to remote
Pull	git pull origin main	Fetch + merge

New branch	git checkout -b feature/x	Create & switch branch
Merge	git merge feature/x	Merge into current
Stash	git stash / git stash pop	Temp save / restore
View log	git log --oneline --graph	Visual commit history
Undo commit	git reset HEAD~1	Undo last commit
Discard changes	git restore filename	Revert file to last commit
Tag release	git tag v1.0.0 && git push --tags	Mark a release version

End of Document
